

Radix Sort

基数排序

▼ 提醒

本页面要介绍的不是 [计数排序](#)。

本页面将简要介绍基数排序。

定义

基数排序（英语：Radix sort）是一种非比较型的排序算法，最早用于解决卡片排序的问题。基数排序将待排序的元素拆分为 k 个关键字，逐一对各个关键字排序后完成对所有元素的排序。

如果是从第 k 关键字到第 1 关键字顺序进行比较，则该基数排序称为 MSD（Most Significant Digit first）基数排序；

如果是从第 1 关键字到第 k 关键字顺序进行比较，则该基数排序称为 LSD（Least Significant Digit first）基数排序。

k - 关键字元素的比较

下面用 k_i 表示元素 x_i 的第 k 关键字。

假如元素有 k 个关键字，对于两个元素 x_i 和 x_j ，默认的比较方法是：

- 比较两个元素的第 k 关键字 k_i 和 k_j ，如果 $k_i < k_j$ 则 $x_i < x_j$ ，如果 $k_i > k_j$ 则 $x_i > x_j$ ，如果 $k_i = k_j$ 则进行下一步；
- 比较两个元素的第 $k-1$ 关键字 $k_{i,k-1}$ 和 $k_{j,k-1}$ ，如果 $k_{i,k-1} < k_{j,k-1}$ 则 $x_i < x_j$ ，如果 $k_{i,k-1} > k_{j,k-1}$ 则 $x_i > x_j$ ，如果 $k_{i,k-1} = k_{j,k-1}$ 则进行下一步；
-
- 比较两个元素的第 1 关键字 $k_{i,1}$ 和 $k_{j,1}$ ，如果 $k_{i,1} < k_{j,1}$ 则 $x_i < x_j$ ，如果 $k_{i,1} > k_{j,1}$ 则 $x_i > x_j$ ，如果 $k_{i,1} = k_{j,1}$ 则 $x_i = x_j$ 。

例子：

- 如果对自然数进行比较，将自然数按个位对齐后往高位补齐，则一个数字从左往右数第 k 位数就可以作为第 k 关键字；
- 如果对字符串基于字典序进行比较，一个字符串从左往右数第 k 个字符就可以作为第 k 关键字；
- C++ 自带的 `std::pair` 与 `std::tuple` 的默认比较方法与上述的相同。

MSD 基数排序

基于 k - 关键字元素的比较方法，可以想到：先比较所有元素的第 k 关键字，就可以确定出各元素大致的大小关系；然后对 **具有相同第 k 关键字的元素**，再比较它们的第 $k-1$ 关键字.....以此类推。

由于是从第 k 关键字到第 1 关键字顺序进行比较，由上述思想导出的排序算法称为 MSD（Most Significant Digit first）基数排序。

算法流程

将待排序的元素拆分为 k 个关键字，先对第 k 关键字进行稳定排序，然后对于每组 **具有相同关键字的元素** 再对第 $k-1$ 关键字进行稳定排序（递归执行）.....最后对于每组 **具有相同关键字的元素** 再对第 1 关键字进行稳定排序。

一般而言，我们默认基数排序是稳定的，所以在 MSD 基数排序中，我们也仅仅考虑借助 **稳定算法**（通常使用计数排序）完成内层对关键字的排序。

正确性参考上文 k - 关键字元素的比较。

参考代码

对自然数排序

下面是使用迭代式 MSD 基数排序对 `unsigned int` 范围内元素进行排序的 C++ 参考代码，可调整 `logW` 和 `W` 的值（建议将 `logW` 设为 32 以便位运算优化）。

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <stack>
4 #include <tuple>
5 #include <vector>
6
7 using std::copy; // from <algorithm>
8 using std::make_tuple;
9 using std::stack;
10 using std::tie;
11 using std::tuple;
12 using std::vector;
13
14 using u32 = uint32_t;
15 using u64 = uint64_t;
16 using u32ptr = u32*;
17
18 void MSD_radix_sort(u32ptr first, u32ptr last) {
19     static constexpr u32 maxlogW = 32; // = log_2 W
20     static constexpr u64 maxW = (u64)1 << maxlogW;
21
22     static constexpr u32 logW = 8;
23     static constexpr u32 W = 1 << logW; // 计数排序的值域
24     static constexpr u32 mask = W - 1; // 用位运算替代取模，详见下面的 key 函数
25
26     u32ptr tmp =
27         (u32ptr)calloc(last - first, sizeof(u32)); // 计数排序用的输出空间
28
29     using node = tuple<u32ptr, u32ptr, u32>;
30     stack<node, vector<node>>> s;
31     s.push(make_tuple(first, last, maxlogW - logW));
32
33     while (!s.empty()) {
34         u32ptr begin, end;
35         u32 shift;
36         size_t length;
37
38         tie(begin, end, shift) = s.top();
39         length = end - begin;
40         s.pop();
41
42         if (begin + 1 >= end) continue; // elements <= 1
43
44         // 计数排序
45         u32 cnt[W] = {};
46         auto key = [](const u32 x, const u32 shift) { return (x >> shift) & mask; };
47
48         for (u32ptr it = begin; it != end; ++it) ++cnt[key(*it, shift)];
49         for (u32 value = 1; value < W; ++value) cnt[value] += cnt[value - 1];
50
51         // 求完前缀和后，计算相同关键字的元素范围
52         if (shift >= logW) {
53             s.push(make_tuple(begin, begin + cnt[0], shift - logW));
54             for (u32 value = 1; value < W; ++value)
```

```

55         s.push(make_tuple(begin + cnt[value - 1], begin + cnt[value],
56                             shift - logW));
57     }
58
59     u32ptr it = end;
60     do {
61         --it;
62         --cnt[key(*it, shift)];
63         tmp[cnt[key(*it, shift)]] = *it;
64     } while (it != begin);
65
66     copy(tmp, tmp + length, begin);
67 }
68 free(tmp);
69 }

```

对字符串排序

下面是使用迭代式 MSD 基数排序对 [空终止字节字符串](#) 基于字典序进行排序的 C++ 参考代码：

```

1 #include <LTalgorithm>
2 #include <LTstack>
3 #include <LTtuple>
4 #include <LTvector>
5
6 using std::copy; // from <LTalgorithm>
7 using std::make_tuple;
8 using std::stack;
9 using std::tie;
10 using std::tuple;
11 using std::vector;
12
13 using NTBS = char*; // 空终止字节字符串
14 using NTBSptr = NTBS*;
15
16 void MSD_radix_sort(NTBSptr first, NTBSptr last) {
17     static constexpr size_t W = 128;
18     static constexpr size_t logW = 7;
19     static constexpr size_t mask = W - 1;
20
21     NTBSptr tmp = (NTBSptr) calloc(last - first, sizeof(NTBS));
22
23     using node = tuple<NTBSptr, NTBSptr, size_t>;
24     stack<node, vector<node>>> s;
25     s.push(make_tuple(first, last, 0));
26
27     while (!s.empty()) {
28         NTBSptr begin, end;
29         size_t index, length;
30
31         tie(begin, end, index) = s.top();
32         length = end - begin;
33         s.pop();
34
35         if (begin + 1 >= end) continue; // elements <= 1
36
37         // 计数排序
38         size_t cnt[W] = {};
39         auto key = [](const NTBS str, const size_t index) { return str[index]; };
40
41         for (NTBSptr it = begin; it != end; ++it) ++cnt[key(*it, index)];
42         for (char ch = 1; value < W; ++value) cnt[ch] += cnt[ch - 1];
43
44         // 求完前缀和后, 计算相同关键字的元素范围
45         // 对于 NTBS, 如果此刻末尾的字符是 \0 则说明这两个字符串相等, 不必继续迭代
46         for (char ch = 1; ch < W; ++ch)
47             s.push(make_tuple(begin + cnt[ch - 1], begin + cnt[ch], index + 1));
48
49         NTBSptr it = end;
50         do {
51             --it;
52             --cnt[key(*it, index)];
53             tmp[cnt[key(*it, index)]] = *it;
54         } while (it != begin);
55
56         copy(tmp, tmp + length, begin);
57     }
58
59     free(tmp);
60 }

```

由于两个字符串的比较很容易冲上 的线性复杂度, 因此在字符串排序这件事情上, MSD 基数排序比大多数基于比较的排序算法在时间复杂度和实际用时上都更加优秀。

与桶排序的关系

前置知识：[桶排序](#)

桶排序需要其它的排序算法来完成对每个桶内部元素的排序。但实际上，完全可以对每个桶继续执行桶排序，直至某一步桶的元素数量。

因此 MSD 基数排序的另一种理解方式是：使用桶排序实现的桶排序。

也因此，可以提出 MSD 基数排序在时间常数上的一种优化方法：假如到某一步桶的元素数量（是自己选的常数），则直接执行插入排序然后返回，降低递归次数。

LSD 基数排序

MSD 基数排序从第 关键字到第 关键字顺序进行比较，为此需要借助递归或迭代来实现，时间常数还是较大，而且在比较自然数上还是略显不便。

而将递归的操作反过来：从第 关键字到第 关键字顺序进行比较，就可以得到 LSD（Least Significant Digit first）基数排序，不使用递归就可以完成的排序算法。

算法流程

将待排序的元素拆分为 个关键字，然后先对 所有元素 的第 关键字进行稳定排序，再对 所有元素 的第 关键字进行稳定排序，再对 所有元素 的第 关键字进行稳定排序.....最后对 所有元素 的第 关键字进行稳定排序，这样就完成了对整个待排序序列的稳定排序。

329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

LSD 基数排序也需要借助一种 **稳定算法** 完成内层对关键字的排序。同样的，通常使用计数排序来完成。

LSD 基数排序的正确性可以参考 [《算法导论（第三版）》第 8.3-3 题的解法](#) 或参考下面的解释：

正确性

回顾一下 k - 关键字元素的比较方法，

- 假如想通过 和 就比较出两个元素 和 的大小，则需要提前知道通过比较 和 得到的结论，以便于应对 的情况；
- 而想通过 和 就比较出两个元素 和 的大小，则需要提前知道通过比较 和 得到的结论，以便于应对 的情况；
-
- 而想通过 和 就比较出两个元素 和 的大小，则需要提前知道通过比较 和 得到的结论，以便于应对 的情况；
- 和 可以直接比较。

现在，将顺序反过来：

- 和 可以直接比较；
- 而知道通过比较 和 得到的结论后，就可以得到比较 和 的结论；
-

- 而知道通过比较 和 得到的结论后，就可以得到比较 和 的结论；
- 而知道通过比较 和 得到的结论后，就最终得到了比较 和 的结论。

在这个过程中，对每个关键字边比较边重排元素的顺序，就得到了 LSD 基数排序。

伪代码

参考代码

下面是使用 LSD 基数排序实现的对 k - 关键字元素的排序。

```

1 constexpr int N = 100010;
2 constexpr int W = 100010;
3 constexpr int K = 100;
4
5 int n, w[K], k, cnt[W];
6
7 struct Element {
8     int key[K];
9
10    bool operator<(const Element& y) const {
11        // 两个元素的比较流程
12        for (int i = 1; i <= k; ++i) {
13            if (key[i] == y.key[i]) continue;
14            return key[i] < y.key[i];
15        }
16        return false;
17    }
18 } a[N], b[N];
19
20 void counting_sort(int p) {
21     memset(cnt, 0, sizeof(cnt));
22     for (int i = 1; i <= n; ++i) ++cnt[a[i].key[p]];
23     for (int i = 1; i <= w[p]; ++i) cnt[i] += cnt[i - 1];
24     // 为保证排序的稳定性，此处循环i应从n到1
25     // 即当两元素关键字的值相同时，原先排在后面的元素在排序后仍应排在后面
26     for (int i = n; i >= 1; --i) b[cnt[a[i].key[p]]--] = a[i];
27     memcpy(a, b, sizeof(a));
28 }
29
30 void radix_sort() {
31     for (int i = k; i >= 1; --i) {
32         // 借助计数排序完成对关键字的排序
33         counting_sort(i);
34     }
35 }

```

实际上并非必须从后往前枚举才是稳定排序，只需对 cnt 数组进行等价于 `std::exclusive_scan` 的操作即可。

▼ 例题 [洛谷 P1177【模板】快速排序](#)

给出 n 个正整数，从小到大输出。

```

1 #include <algorithm>
2 #include <iostream>
3 #include <utility>
4
5 void radix_sort(int n, int a[]) {
6     int *b = new int[n]; // 临时空间
7     int *cnt = new int[1 << 8];
8     int mask = (1 << 8) - 1;
9     int *x = a, *y = b;
10    for (int i = 0; i < 32; i += 8) {
11        for (int j = 0; j != (1 << 8); ++j) cnt[j] = 0;
12        for (int j = 0; j != n; ++j) ++cnt[x[j] >> i & mask];
13        for (int sum = 0, j = 0; j != (1 << 8); ++j) {
14            // 等价于 std::exclusive_scan(cnt, cnt + (1 << 8), cnt, 0);
15            sum += cnt[j], cnt[j] = sum - cnt[j];
16        }
17        for (int j = 0; j != n; ++j) y[cnt[x[j] >> i & mask]++] = x[j];
18        std::swap(x, y);
19    }
20    delete[] cnt;
21    delete[] b;
22 }
23
24 int main() {
25     std::ios::sync_with_stdio(false);
26     std::cin.tie(nullptr);
27     int n;
28     std::cin >> n;
29     int *a = new int[n];
30     for (int i = 0; i < n; ++i) std::cin >> a[i];
31     radix_sort(n, a);
32     for (int i = 0; i < n; ++i) std::cout << a[i] << ' ';
33     delete[] a;
34     return 0;
35 }

```

性质

稳定性

如果对内层关键字的排序是稳定的，则 MSD 基数排序和 LSD 基数排序都是稳定的排序算法。

时间复杂度

通常而言，基数排序比基于比较的排序算法（比如快速排序）要快。但由于需要额外的内存空间，因此当内存空间稀缺时，原地置换算法（比如快速排序）或许是个更好的选择。¹

一般来说，如果每个关键字的值域都不大，就可以使用 [计数排序](#) 作为内层排序，此时的复杂度为 $O(n \cdot k)$ ，其中 k 为第 i 关键字的值域大小。如果关键字值域很大，就可以直接使用基于比较的排序而无需使用基数排序了。

空间复杂度

MSD 基数排序和 LSD 基数排序的空间复杂度都为 $O(n)$ 。

参考资料与注释

-
1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein *Introduction to Algorithms* (3rd ed.). MIT Press and McGraw-Hill, 2009. ISBN 978-0-262-03384-8. "8.3 Radix sort", pp. 199. [↩](#)

本页面最近更新：2024/11/10 20:22:04, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [NachtgeistW](#), [Alisahhh](#), [countercurrent-time](#), [Early0v0](#), [Enter-tainer](#), [H-J-Granger](#), [hly1204](#), [iamtwz](#), [lr1d](#), [Konano](#), [ksyx](#), [OhGaGaGaGa](#), [ouuan](#), [partychicken](#), [sbofgayschool](#), [SukkaW](#), [Tiphereth-A](#), [TrickEye](#), [untitledunrevised](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用